

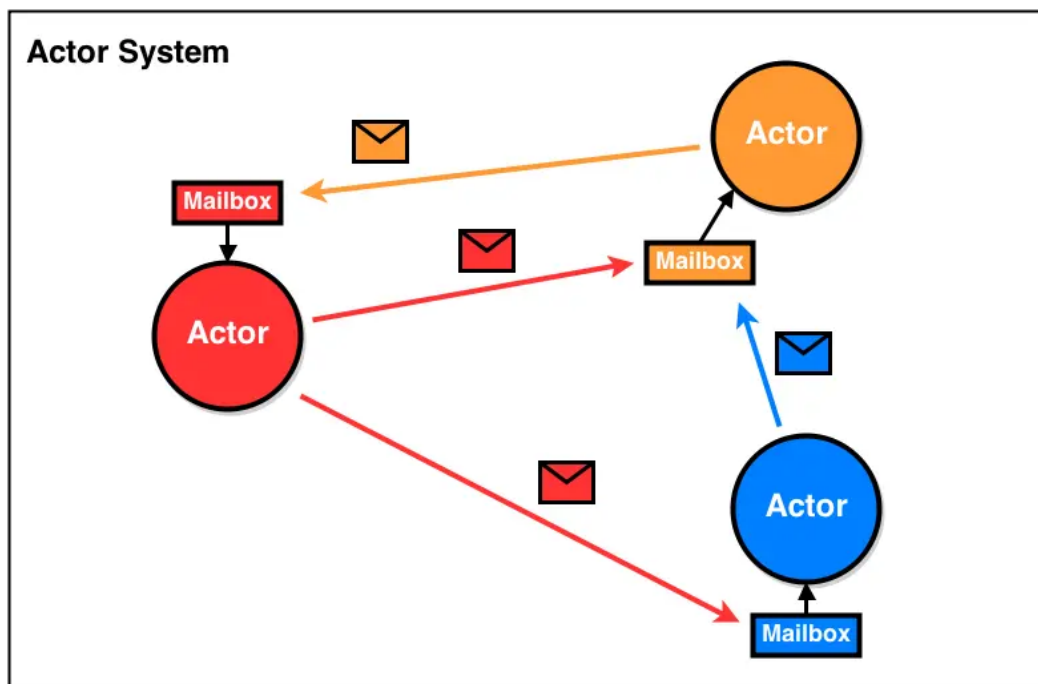
20250724日报

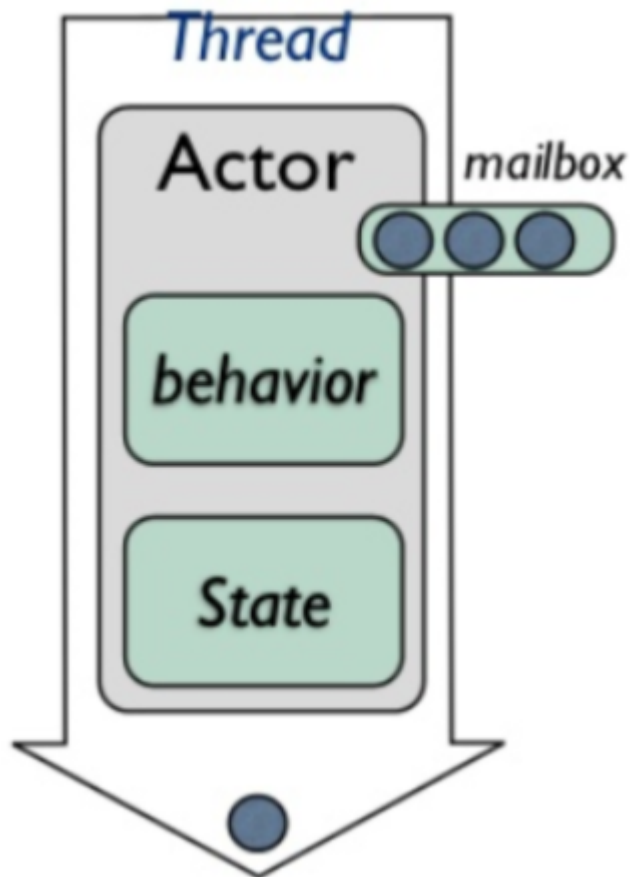
今日工作内容

1. 系统调研了Actor模型和常用的基于虚拟Actor的框架；

a. Actor模型

- i. Actor是以单线程存在的，所有消息都是顺序到达的，每次收到消息后，就放入队列，而它每次也从队列中取出消息来处理；
- ii. 每一个Actor有一个id和它对应，一个id对应的Actor只会在集群中存在一个，使用者只需要通过id就能随时访问不需要关注改Actor在集群的什么位置；
- iii. 每一个Actor看作是一个独立的实体，拥有自己独立的状态。在Actor模型中，所有的操作都是基于消息传递实现的，这种方式可以避免竞态条件和死锁等并发问题；
- iv. Actor之间的通信是异步的，也就是说，发送消息的Actor不需要等待接收消息的Actor处理完消息才能继续执行；
- v. Actor模型=状态（在内存、已持久化）+行为（class 或 struct（go中））+消息（Actor之间通信的数据结构）；



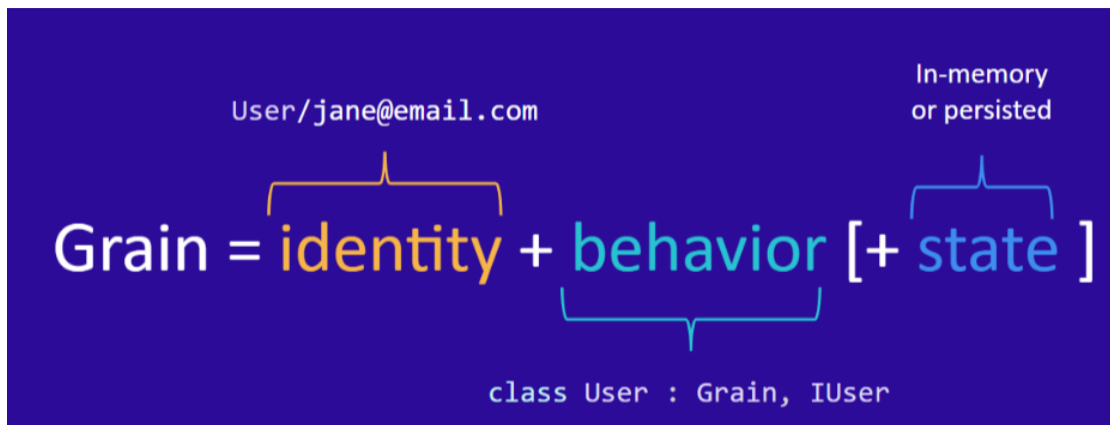


b. Orleans

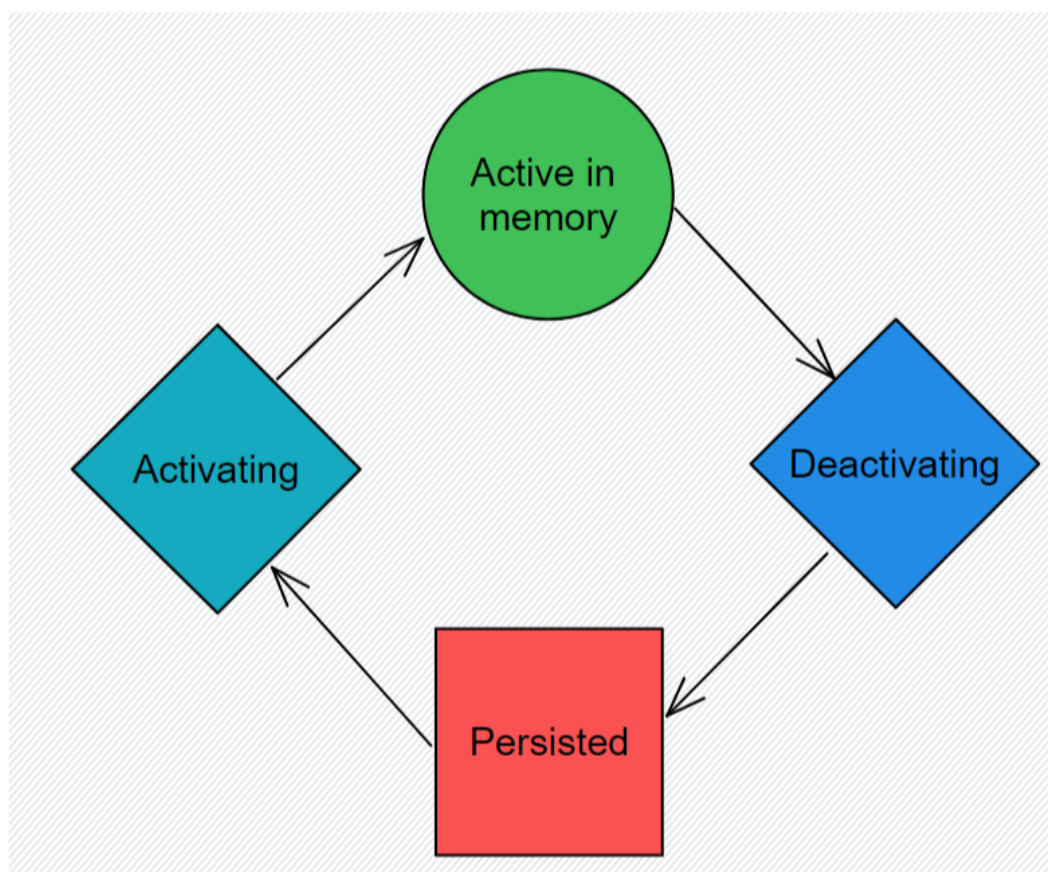
- i. Orleans被认为是Actor模型的分布式版本，是一种改进的Actor模型。在Orleans中，Actors被称作Grains，采用接口来表示，Actors的消息用异步方法来接收；
- ii. Grains：它是Orleans应用程序的业务逻辑实现与抽象，Grains是彼此孤立的原子单位，分布的，持久的；
- iii. Silo：它是一个主机服务，里面主要用于执行Grains，也就是说Grains开发完成后需要注册到Silo中，然后等待调用。它监听一个端口，用来监听从Silo到Silo的消息或者从客户端到Silo的消息的；
- iv. Cluster：大量的Silo同时在一起工作就形成了Orleans的集群；

c. Grain

- i. Grain是一个虚拟执行组件，任何Orleans应用程序中的基本构建块都是一个Grain。Grain由用户定义的标识、行为和状态组成的实体，如：

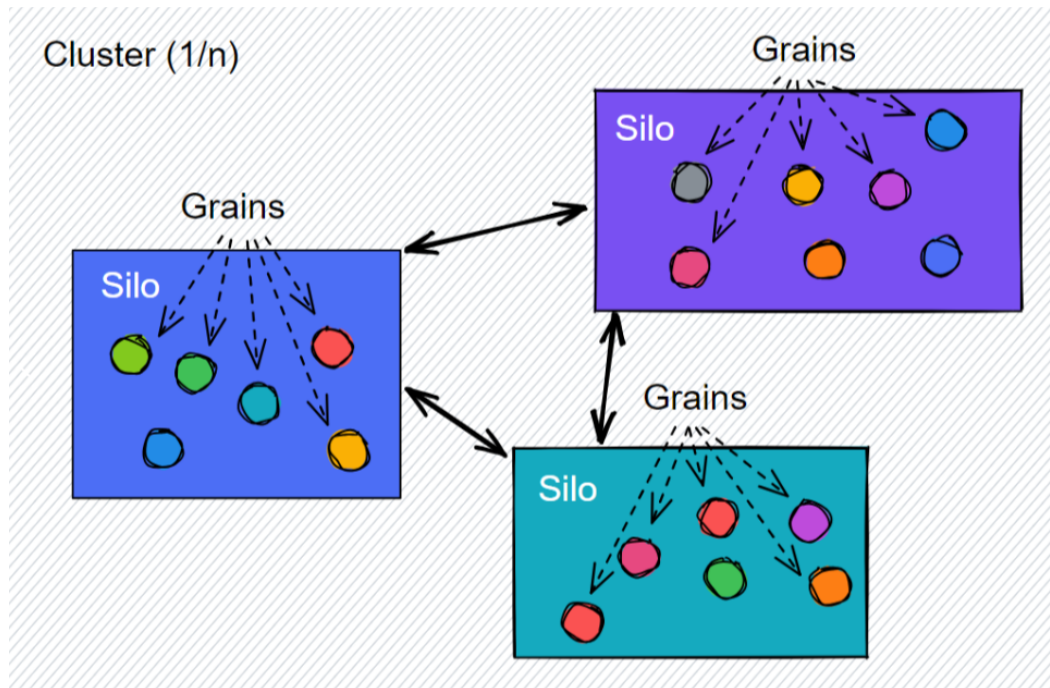


- ii. 虚拟化抽象：上面的Grain的逻辑标识与物理位置完全解耦。通过逻辑标识获取Grain引用，无需关心其物理位置或激活状态，运行时自动管理Grain的激活与停用，使Grain在逻辑上始终存在（换句话说，使用时无需关注Actor实例的创建、销毁，可以直接调用Actor提供的方法）；
- iii. 按需激活：当Grain首次被调用时，Orleans运行时会自动执行：激活（从持久化存储加载状态到内存，创建实例）->执行请求->停用（闲置超时后，自动将状态存回存储并释放内存资源）；
- iv. 位置透明性：Grain引用时仅使用逻辑标识即可，运行时会自动解析物理位置：若Grain未激活，将其分配到任一可用Silo并激活->若Silo故障，Grain自动在其他节点重新激活；
- v. 生命周期自动化：Orleans运行时通过GrainLifecycle阶段管理状态加载、资源初始及清理，其生命周期转换如下：



d. Silo

- i. Silo是Grain的物理宿主，负责其生命周期管理（激活、停用、持久化、活跃）和消息调度，每个Silo可托管数百万个Grain实例，通过单线程模型保证每个Grain的调用串行化，避免并发冲突；
- ii. 多个Silo组成集群，共同提供分布式计算能力。集群通过动态增减Silo实现水平扩展，并通过心跳机制检测节点故障，自动迁移Grain；



e. 比较.NET四种流行Actor框架

Akka.Net

1. 基于传统Actor模型，移植自Scala/Java的Akka，强调监督层次结构和显式Actor生命周期管理；
2. 需显式定义父子Actor关系及故障恢复策略，需手动实例化；

Proto.Actor/Dapr/Orleans

1. 基于虚拟Actor模型：位置透明（无需感知物理节点位置）、按需激活与自动回收、状态与生命周期分离、跨节点容错（节点故障，自动迁移至健康节点）；
2. Orleans是微软支持的项目，有着全面的文档、庞大的社区；
3. Proto.Actor未达到1.0版本，API可能破坏性变更，并且社区较小；
4. 对于开发者，Proto.Actor未达1.0版本，API可能破坏性变更；
5. 对于开发者，Dapr需Sidecar模式编排，本地调试繁琐；
6. 对于开发者，Orleans的隐式逻辑可能会增加问题的定位难度；

2. 初步学习性能分析工具——火焰图；

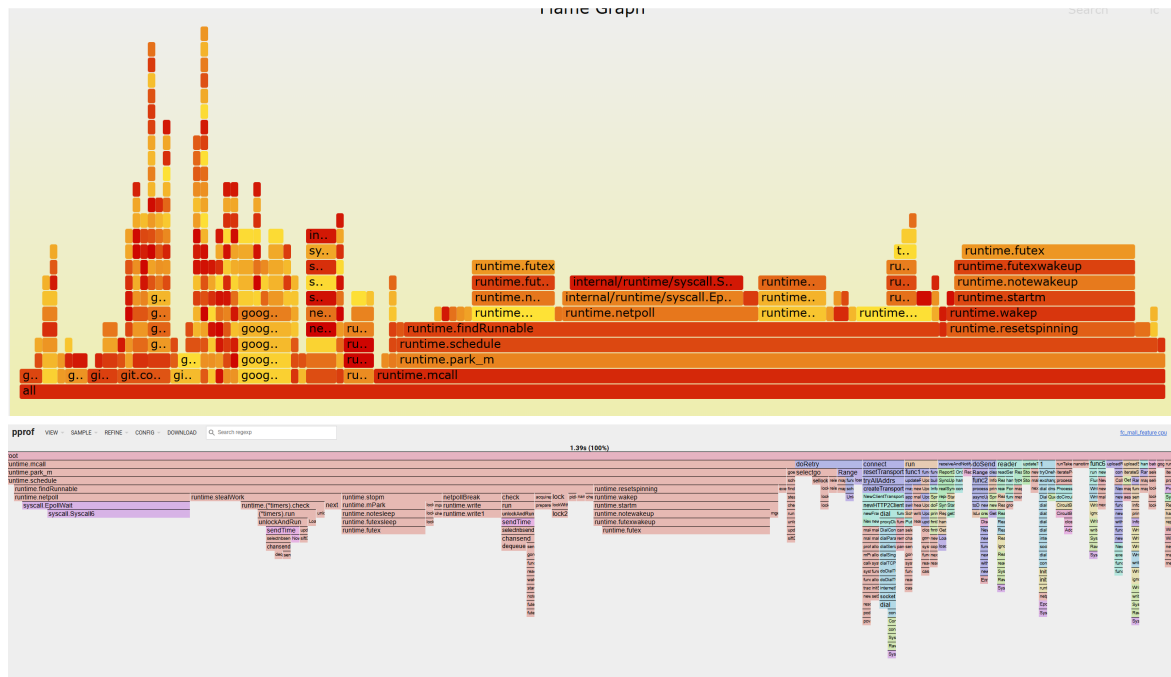
- a. 定义：火焰图将程序运行时的函数调用栈及其资源占用比例（如CPU时间、内存分配）以层次化图形展示。每个矩形代表一个函数，宽度表示资源占用比例，高度表示

调用栈深度。

b. 查看方法：

- Y轴（垂直）：调用栈深度，底部为入口函数，顶部为当前执行的函数；
- X轴（水平）：表示资源占用比例（非时间顺序），宽度越宽的函数消耗资源越多（如CPU时间、内存分配）；
- 火焰图就是看顶层的哪个函数占据的宽度最大。只要有“平顶”，就表示该函数可能存在性能问题；

go tool pprof -seconds 60 http://21.6.68.166:8091/debug/pprof/profile在dev上初步获得fc_mall_feature服务的的火焰图（还未分析）：



明日待办：

- 继续推进“fc_mall_feature性能优化”需求；